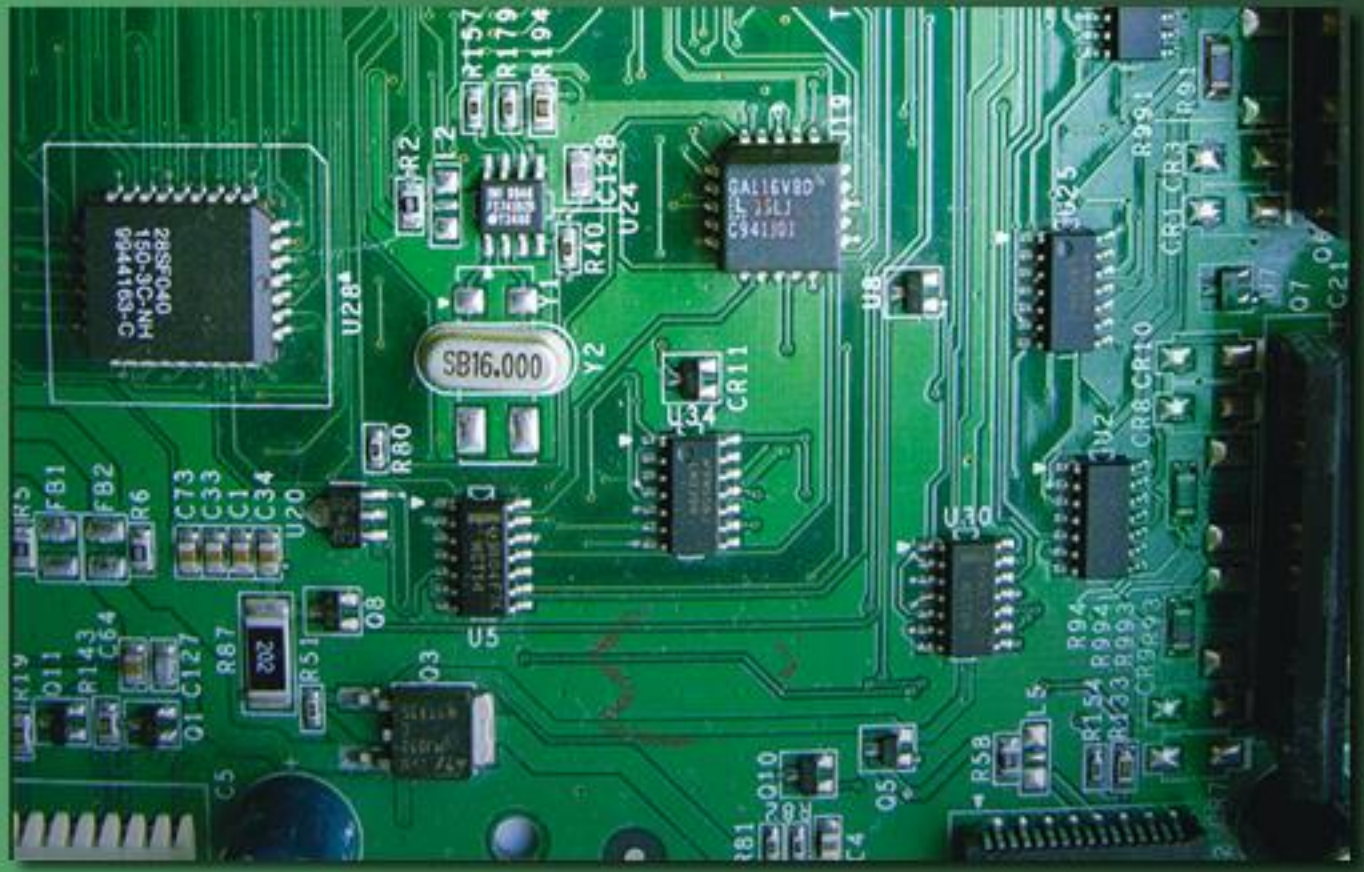
The Intel Microprocessors

8086/8088, 80186/80188, 80286, 80386, 80486 Pentium, Pentium Pro Processor, Pentium II, Pentium 4, and Core2 with 64-bit Extensions

Architecture, Programming, and Interfacing



EIGHTH EDITION

Barry B. Brey

PEARSON

Chapter 13: Direct Memory Access and DMA-Controlled I/O

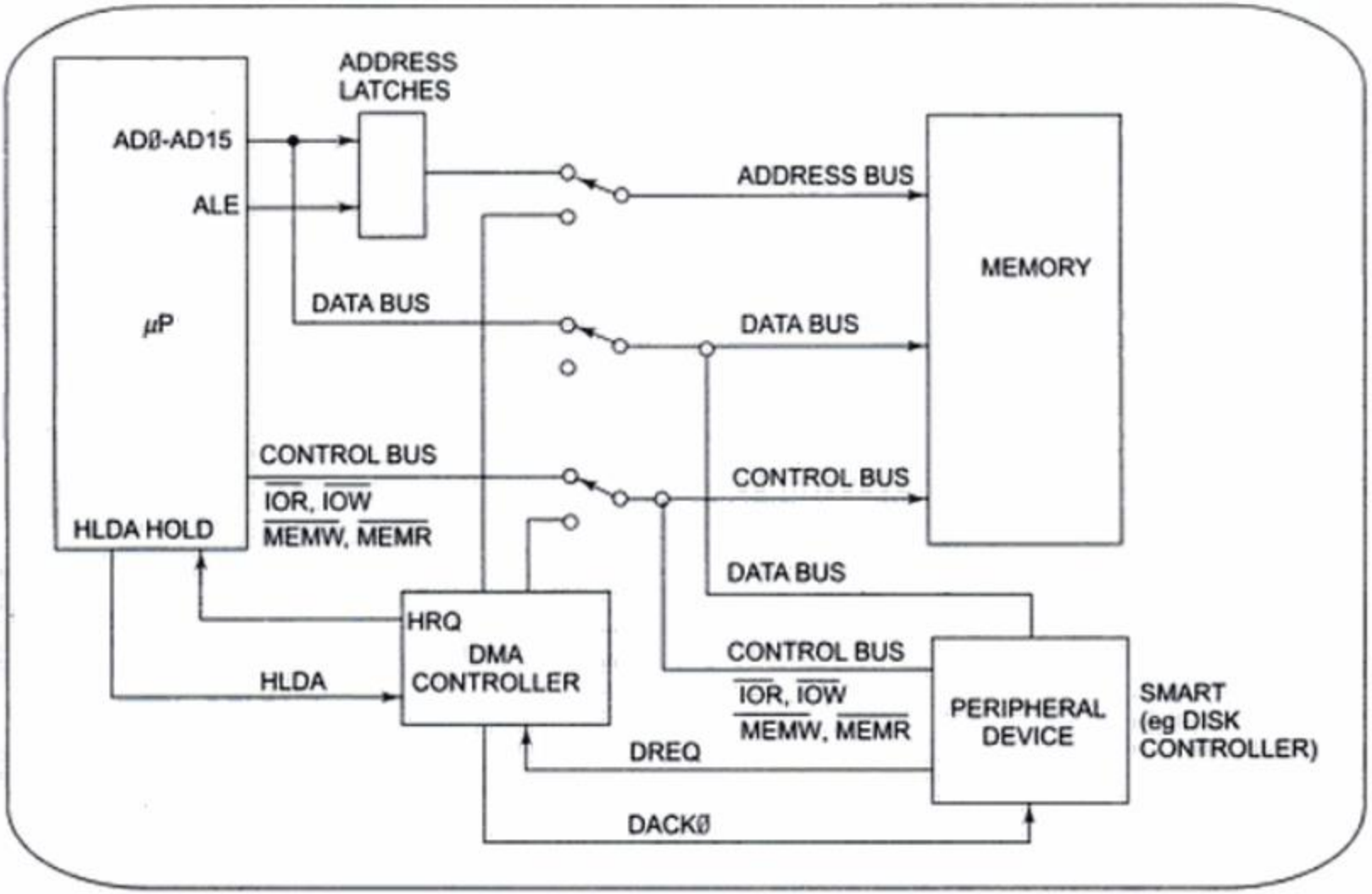
Introduction

- The DMA I/O technique provides direct access to the memory while the microprocessor is temporarily disabled.
- This allows data to be transferred between memory and I/O at a speed limited by the speed of the memory component and the DMA controller (33 to 150 M-bytes transfer rates).
- The common purposes of DMA are **DRAM refresh**, **video displays for refreshing the screen**, **disk memory system write and read** and also **high-speed memory-to-memory transfer**.

Introduction

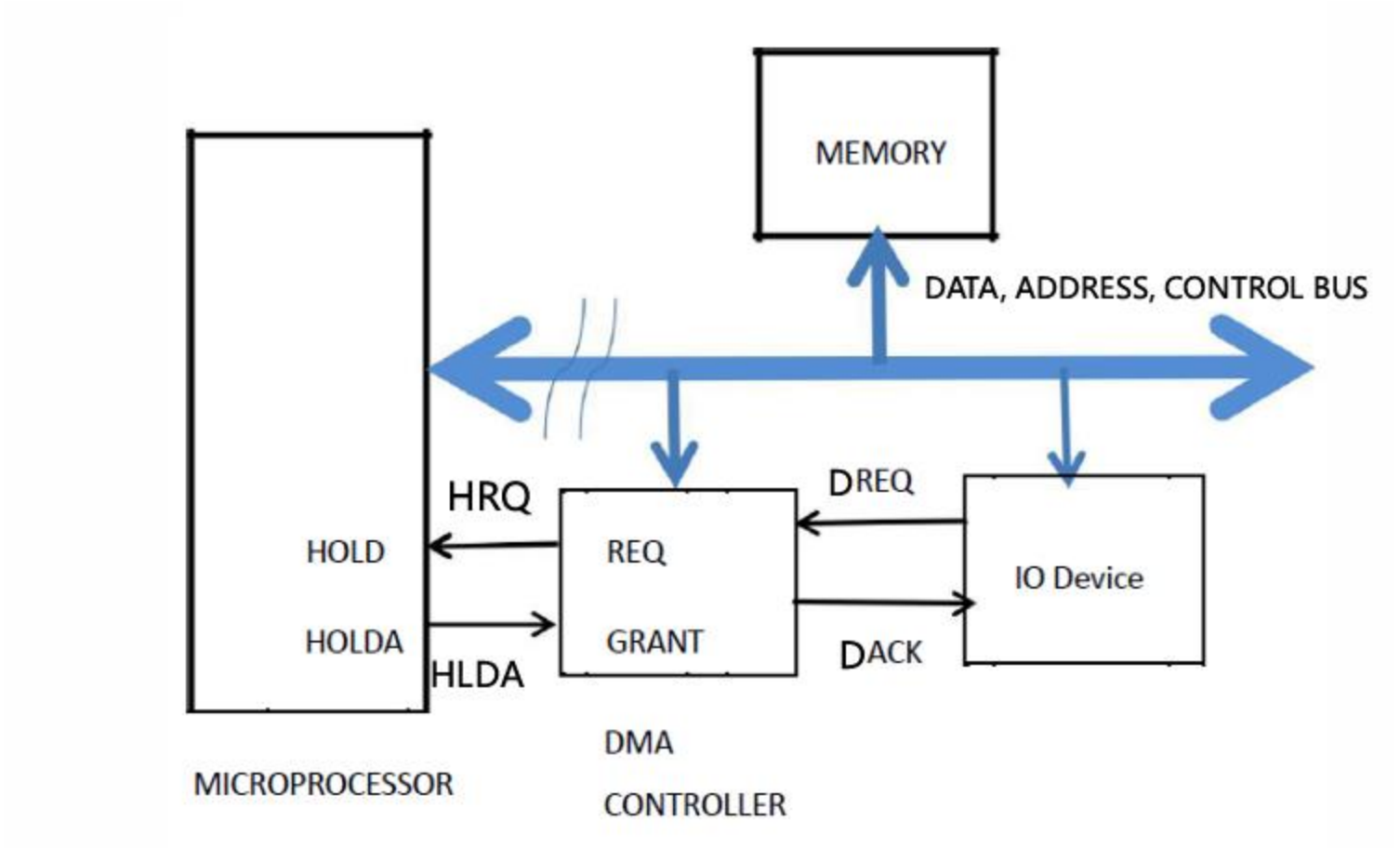
- DMA is designed by Intel to transfer data at the fastest rate.
- It allows the device to **transfer the data directly to/from memory without any interference of the CPU**.
- Using a **DMA controller**, the device requests the CPU to hold its data, address and control bus, so the device is free to transfer data directly to/from the memory.
- The DMA data transfer is initiated only after **receiving HLDA signal from the CPU**.
- **Programmed I/O and Interrupt driven** I/O are the alternative methods of data transferring against Direct Memory Access(DMA)

Figure: Block diagram showing how a DMA controller operates in a microcomputer system.



Ref.- Microprocessors & Interfacing by Douglas V. Hall, 2nd Edition, Pp. 348-351)

Figure: Block diagram showing how a DMA controller operates in a microcomputer system.



BASIC DMA OPERATION

- Two control signals are used to request and acknowledge a direct memory access (DMA) transfer in the **microprocessor-based system.**
  - the HOLD pin is an input used to request a DMA action
  - the HLDA pin is an output that acknowledges the DMA action

BASIC DMA OPERATION

- when the processor recognizes the hold, it stops executing software and enters hold cycles
- HOLD input has higher priority than **INTR or NMI**
- the only microprocessor pin that has a higher priority than a HOLD is the RESET pin

BASIC DMA OPERATION

- Initially, when any device has to send data between the device and the memory, the device has to **send DMA request (DRQ) to DMA controller**.
- The DMA controller **sends Hold request (HRQ) to the CPU** and waits for the CPU to assert the HLDA.
- Then the **microprocessor leaves the contro**l over the data bus, address bus, and control bus; and **acknowledges the HOLD request through HLDA signal.**
- Now the CPU is in HOLD state and the DMA controller has to manage the operations over buses between the CPU, memory, and I/O devices.

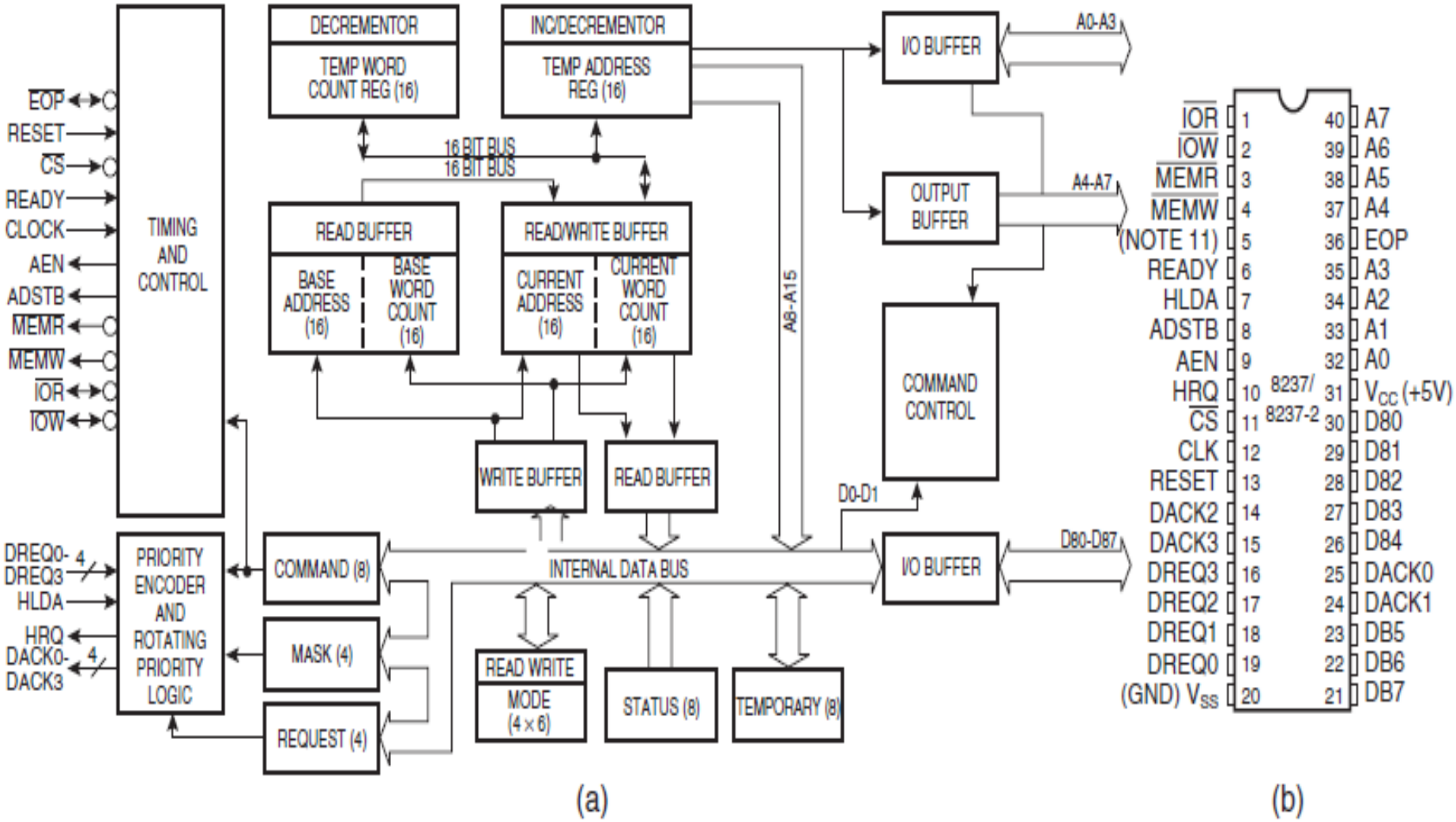
Basic DMA Definitions

- Direct memory accesses normally occur between an I/O device and memory without the use of the microprocessor.
  - a **DMA read** transfers data **from the memory to the I/O device**
  - **A DMA read** causes both the MRDC and IOWC signals to activate simultaneously
  - A **DMA write** transfers data **from an I/O device to memory**
  - **A DMA write causes the** MWTC and IORC signals to both activate
- Memory & I/O are controlled simultaneously.
- The data transfer speed is determined by the speed of the **memory device or a DMA controller** that often controls DMA transfers.

THE 8237 DMA CONTROLLER

- The 8237 supplies memory & I/O with **control signals and memory address** information during the DMA transfer.
  - actually a special-purpose microprocessor whose job is high-speed data transfer between memory and I/O
- It has **four channels,** which can be used over four I/O devices.
- Figure 13–3 shows the pin-out and block diagram of the 8237 programmable DMA controller.

Figure 13–3 The 8237A-5 programmable DMA controller. (a) Block diagram and (b) pin-out. (Courtesy of Intel Corporation.)



THE 8237 DMA CONTROLLER

- 8237 is not a discrete component in modern microprocessor-based systems.
  - it appears within many system controller chip sets
- 8237 is a four-channel device compatible with 8086/8088, adequate for small systems.
  - expandable to any number of DMA channel inputs
- 8237 is capable of DMA transfers at rates up to 1.6M bytes per second.
  - each channel is capable of addressing a full 64K-byte section of memory and transfer up to 64K bytes with a single programming

8237 Internal Registers

CAR

- The **current address register** holds a 16-bit memory address used for the DMA transfer.
  - each channel has its own current address register for this purpose
- When a byte of data is transferred during a DMA operation, CAR is either **incremented or decremented**.
  - depending on how it is programmed

8237 Internal Registers

CWCR

- The **current word count register** programs a channel for the number of bytes (up to 64K) transferred during a DMA action.
- The number loaded into this register is one less than the number of bytes transferred.
  - for example, if a 10 is loaded to CWCR, then 11 bytes are transferred during the DMA action

8237 Internal Registers

BA and BWC

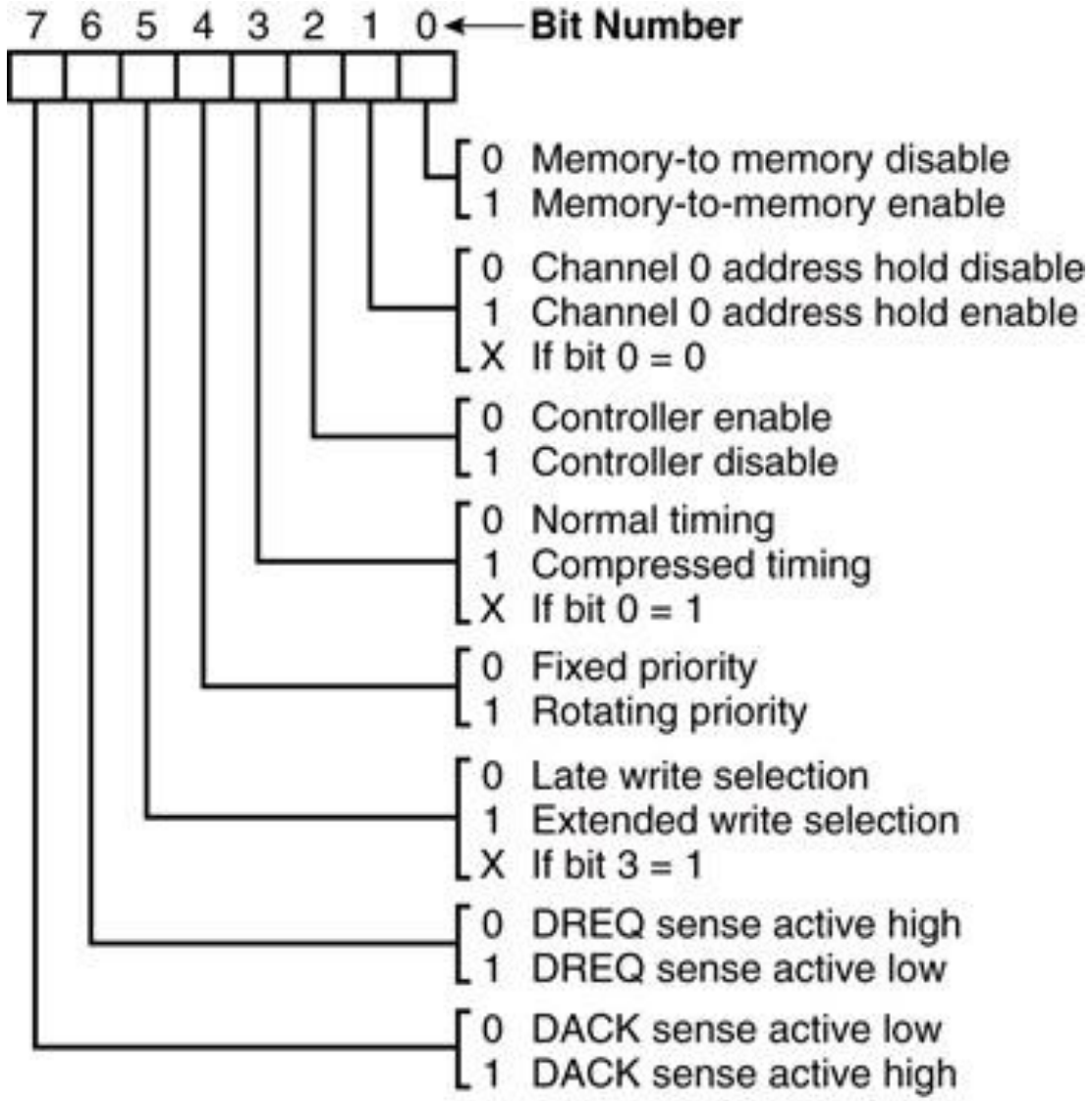
- The **base address** (BA) and **base word count** (BWC) registers are used when auto-initialization is selected for a channel.
- In auto-initialization mode, these registers are used to reload the CAR and CWCR after the DMA action is completed.
  - allows the same count and address to be used to transfer data from the same memory area

8237 Internal Registers

CR

- The **command register** programs the operation of the 8237 DMA controller.
- The register uses bit position 0 to select the memory-to-memory DMA transfer mode.
  - memory-to-memory DMA transfers use DMA **channel 0 to hold the source address**
  - DMA **channel 1 holds** the **destination address**

Figure: 8237A-5 command register. (Courtesy of Intel Corporation.)

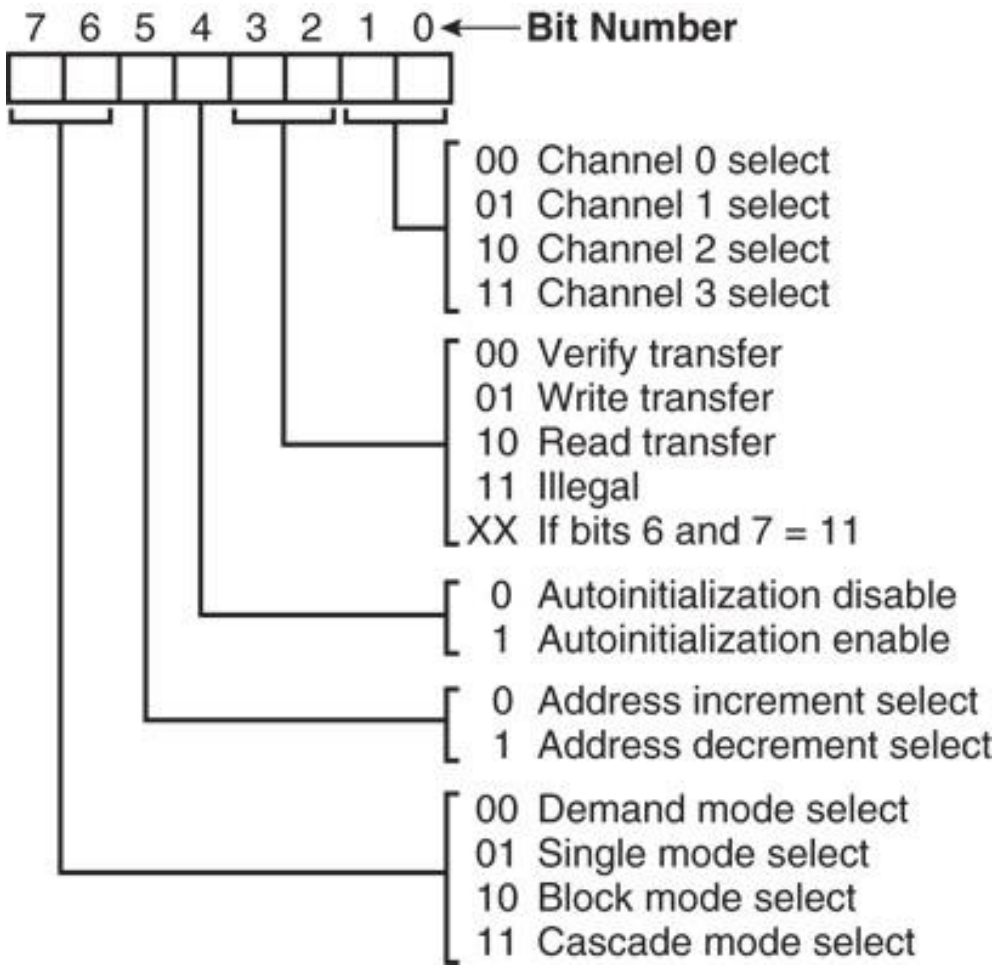


8237 Internal Registers

MR

- The **mode register** programs the mode of operation for a channel.
- Each channel has its **own mode register** as selected by bit positions 1 and 0.
  - remaining bits of the mode register select operation, **auto-initialization, increment/decrement, and mode for the channel**

Figure 13–5 8237A-5 mode register. (Courtesy of Intel Corporation.)

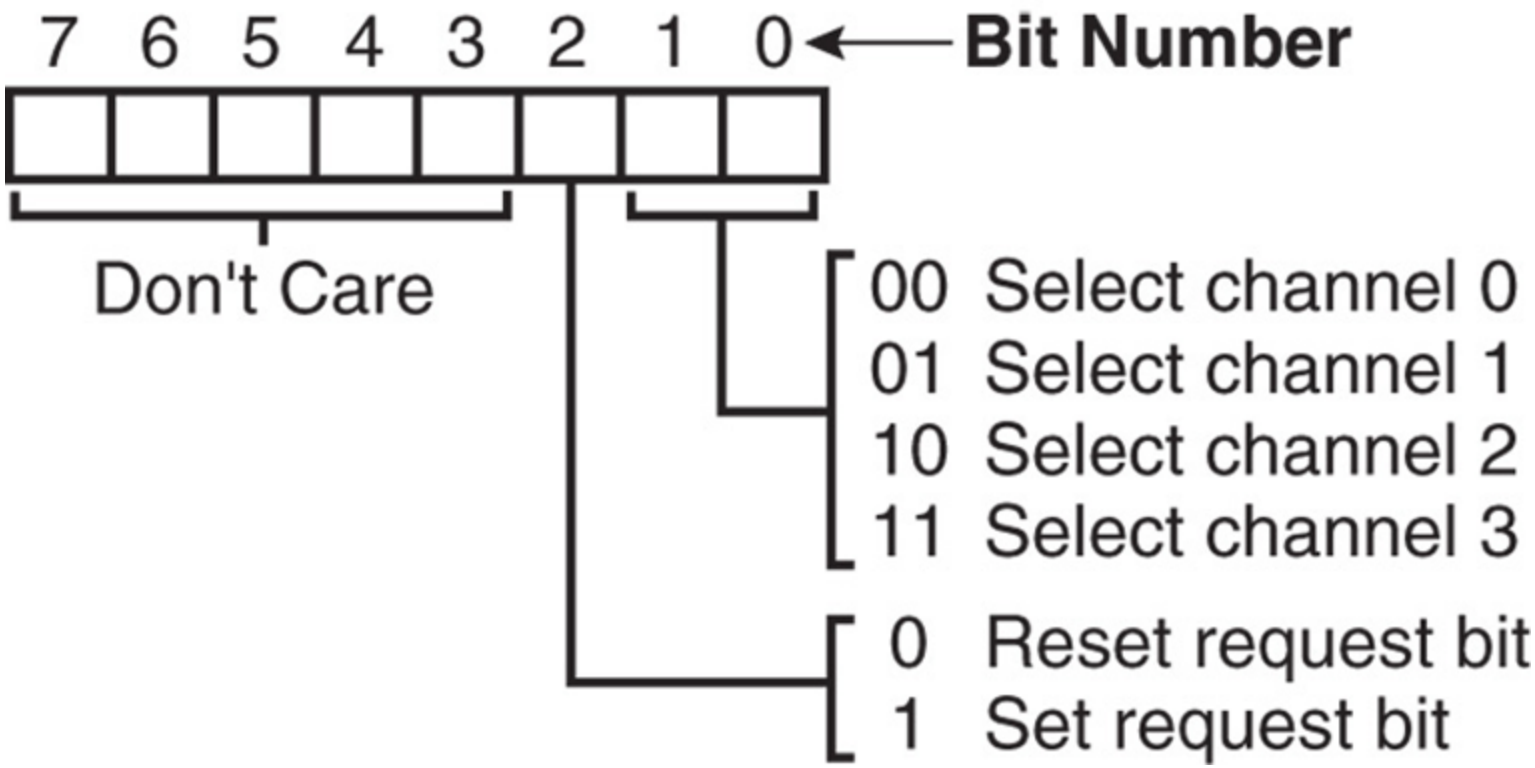


8237 Internal Registers

BR

- The **bus request register** is used to request a DMA transfer via software.
  - very useful in **memory-to-memory** transfers, where an external signal is not available to begin the DMA transfer

Figure 13–6 8237A-5 request register. (Courtesy of Intel Corporation.)

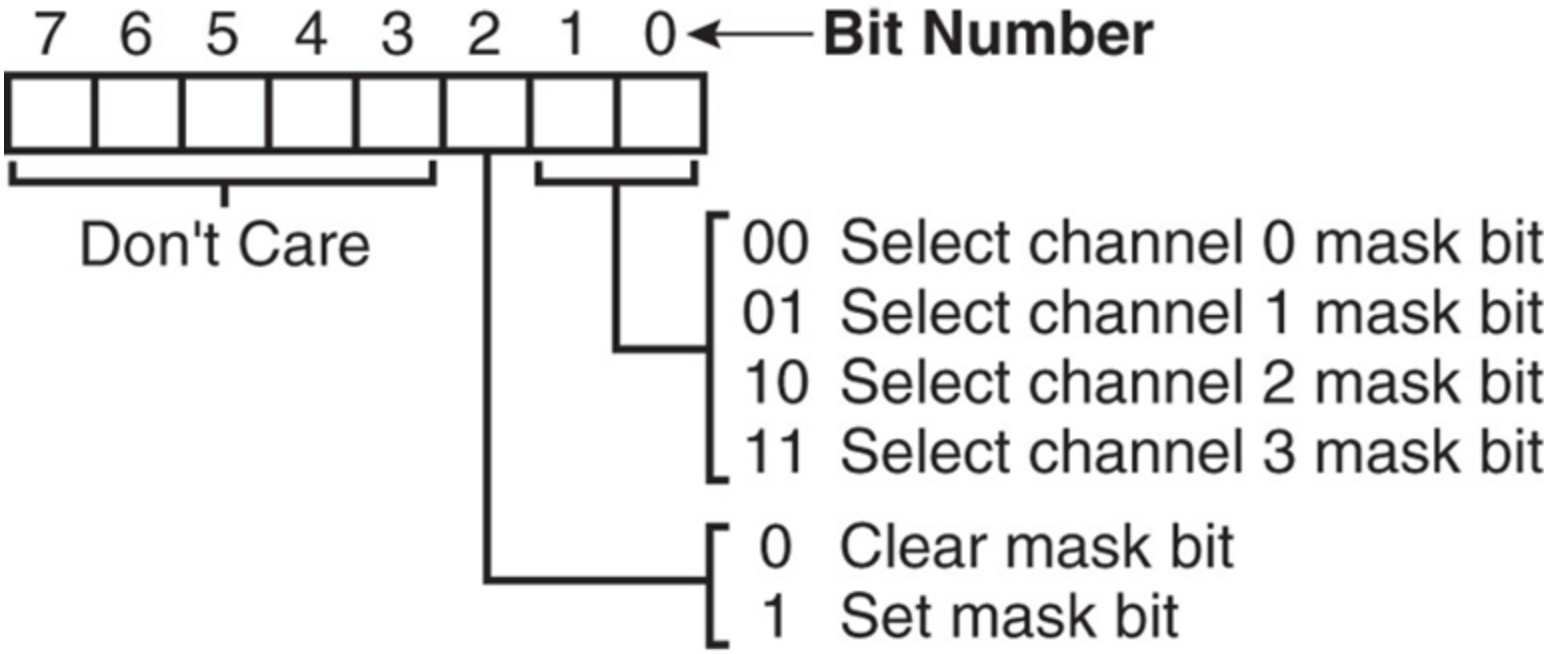


8237 Internal Registers

MRSR

- The **mask register set/reset** sets or clears the channel mask.
  - if the mask is set (1), the **channel is disabled**
  - the **RESET signal** sets **all channel masks to disable them**

Figure 13–7 8237A-5 mask register set/reset mode. (Courtesy of Intel Corporation.)

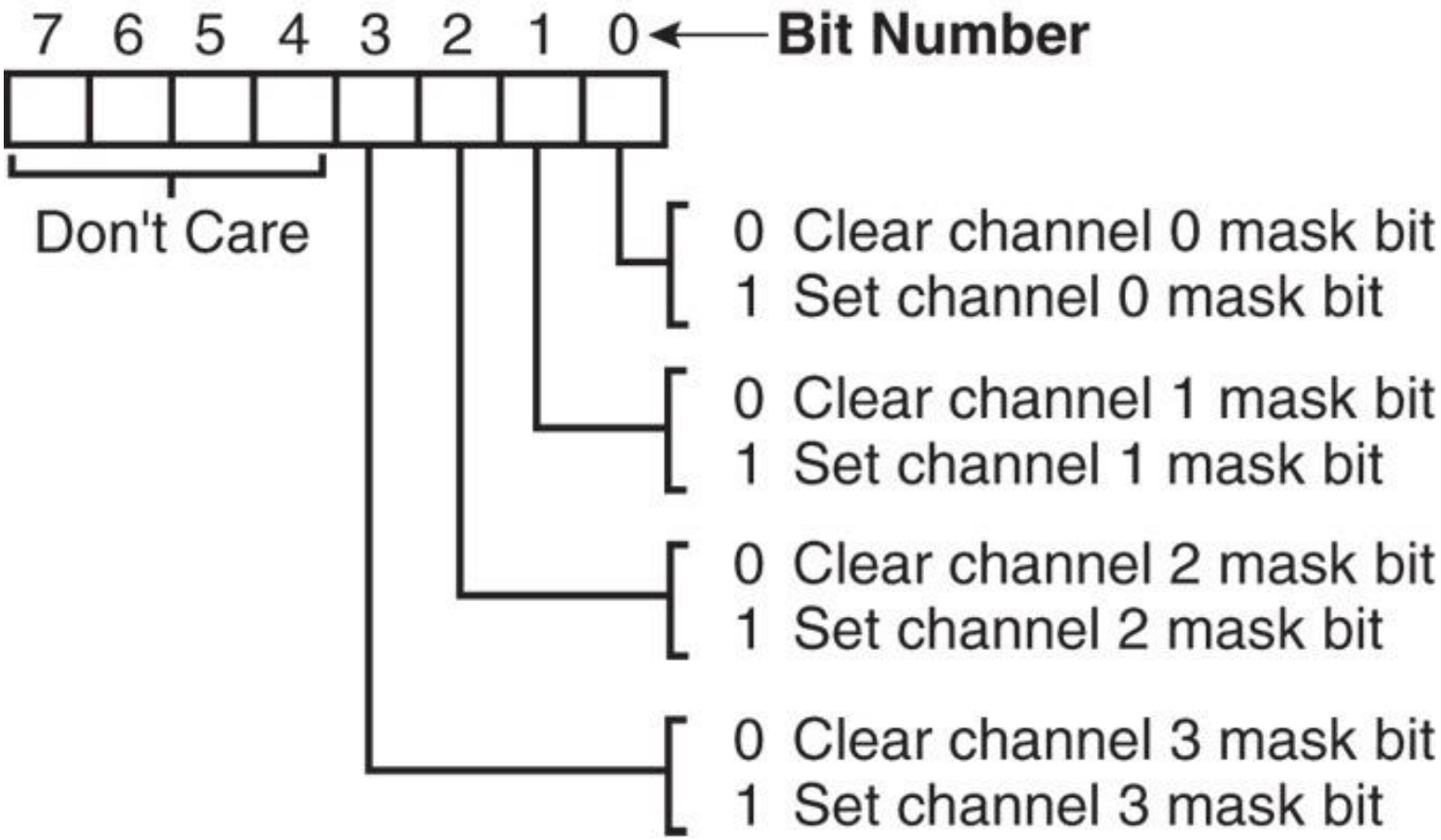


8237 Internal Registers

MSR

- The **mask register** clears or sets all of the masks with one command instead of individual channels, as with the MRSR.

Figure 13–8 8237A-5 mask register. (Courtesy of Intel Corporation.)

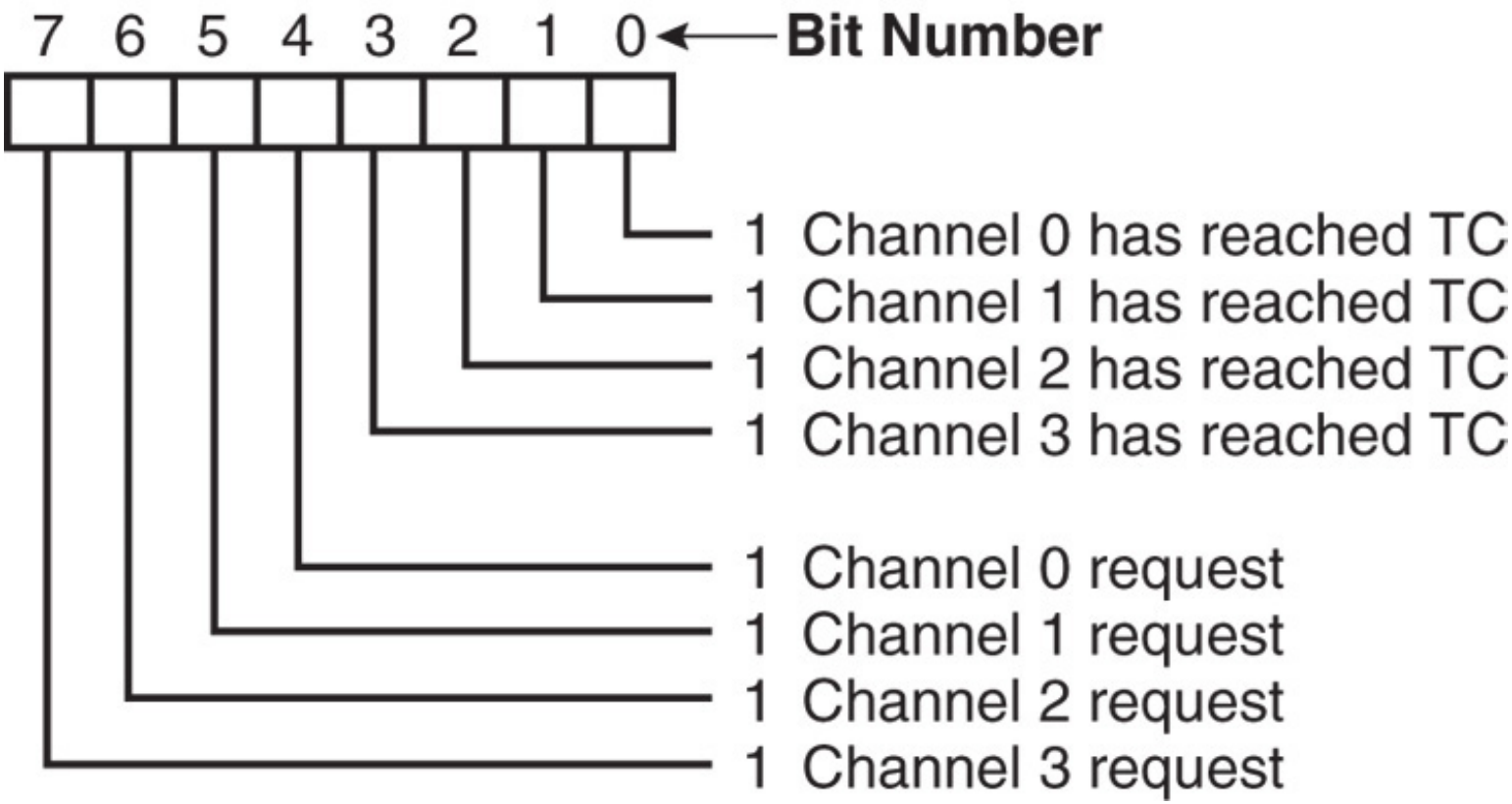


8237 Internal Registers

SR

- The **status register** shows status of **each DMA channel**. The **TC bits** indicate if the channel has reached **its terminal count** (transferred all its bytes).
- When the terminal count is reached, the DMA transfer is terminated for most modes of operation.
  - the **request bits indicate** whether the **DREQ input** for a given channel is **active**

Figure 13–9 8237A-5 status register. (Courtesy of Intel Corporation.)



Software Commands

- **Three software commands** are used to control the operation of the 8237.
- These commands **do not have a binary bit pattern**, as do various control registers within the 8237.
  - **a simple output to the correct port number** enables the software command
- Fig 13–10 shows **I/O port assignments** that access all registers and the software commands.

Figure 13–10 8237A-5 command and control port assignments. (Courtesy of Intel Corporation.)

Signals						Operation
A3	A2	A1	A0	$\overline{IOR}$	$\overline{IOW}$	
1	0	0	0	0	1	Read Status Register
1	0	0	0	1	0	Write Command Register
1	0	0	1	0	1	Illegal
1	0	0	1	1	0	Write Request Register
1	0	1	0	0	1	Illegal
1	0	1	0	1	0	Write Single Mask Register Bit
1	0	1	1	0	1	Illegal
1	0	1	1	1	0	Write Mode Register
1	1	0	0	0	1	Illegal
1	1	0	0	1	0	Clear Byte Pointer Flip/Flop
1	1	0	1	0	1	Read Temporary Register
1	1	0	1	1	0	Master Clear
1	1	1	0	0	1	Illegal
1	1	1	0	1	0	Clear Mask Register
1	1	1	1	0	1	Illegal
1	1	1	1	1	0	Write All Mask Register Bits

Software Commands

Master clear

- Acts exactly the same as the RESET signal to the 8237.
  - as with the RESET signal, this command disables all channels

Clear mask register

- Enables all four DMA channels.

Software Commands

Clear the first/last flip-flop

- Clears the first/last (F/L) flip-flop within 8237.
- The F/L flip-flop selects which byte (low or high order) is read/written in the current address and current count registers.
  - if F/L = 0, the low-order byte is selected
  - if F/L = 1, the high-order byte is selected
- Any read or write to the address or count register automatically toggles the F/L flip-flop.

Programming the Address and Count Registers

- Figure 13–11 shows I/O port locations for programming the count and address registers for each channel.
- The state of the **F/L flip-flop determines whether the LSB or MSB is programmed.**
  - if the state is unknown, count and address could be programmed incorrectly
- It is important to disable the DMA channel before the address and count are programmed.

Figure 13–11 8237A-5 DMA channel I/O port addresses. (Courtesy of Intel Corporation.)

Channel	Register	Operation	Signals							Internal Flip-Flop	Data Bus DB0-DB7
			$\overline{CS}$	$\overline{IOR}$	$\overline{IOW}$	A3	A2	A1	A0		
0	Base and Current Address	Write	0	1	0	0	0	0	0	0	A0-A7
			0	1	0	0	0	0	0	1	A8-A15
	Current Address	Read	0	0	1	0	0	0	0	0	A0-A7
			0	0	1	0	0	0	0	1	A8-A15
	Base and Current Word Count	Write	0	1	0	0	0	0	1	0	W0-W7
			0	1	0	0	0	0	1	1	W8-W15
	Current Word Count	Read	0	0	1	0	0	0	1	0	W0-W7
			0	0	1	0	0	0	1	1	W8-W15
1	Base and Current Address	Write	0	1	0	0	0	1	0	0	A0-A7
			0	1	0	0	0	1	0	1	A8-A15
	Current Address	Read	0	0	1	0	0	1	0	0	A0-A7
			0	0	1	0	0	1	0	1	A8-A15
	Base and Current Word Count	Write	0	1	0	0	0	1	1	0	W0-W7
			0	1	0	0	0	1	1	1	W8-W15
	Current Word Count	Read	0	0	1	0	0	1	1	0	W0-W7
			0	0	1	0	0	1	1	1	W8-W15
2	Base and Current Address	Write	0	1	0	0	1	0	0	0	A0-A7
			0	1	0	0	1	0	0	1	A8-A15
	Current Address	Read	0	0	1	0	1	0	0	0	A0-A7
			0	0	1	0	1	0	0	1	A8-A15
	Base and Current Word Count	Write	0	1	0	0	1	0	1	0	W0-W7
			0	1	0	0	1	0	1	1	W8-W15
	Current Word Count	Read	0	0	1	0	1	0	1	0	W0-W7
			0	0	1	0	1	0	1	1	W8-W15
3	Base and Current Address	Write	0	1	0	0	1	1	0	0	A0-A7
			0	1	0	0	1	1	0	1	A8-A15
	Current Address	Read	0	0	1	0	1	1	0	0	A0-A7
			0	0	1	0	1	1	0	1	A8-A15
	Base and Current Word Count	Write	0	1	0	0	1	1	1	0	W0-W7
			0	1	0	0	1	1	1	1	W8-W15
	Current Word Count	Read	0	0	1	0	1	1	1	0	W0-W7
			0	0	1	0	1	1	1	1	W8-W15

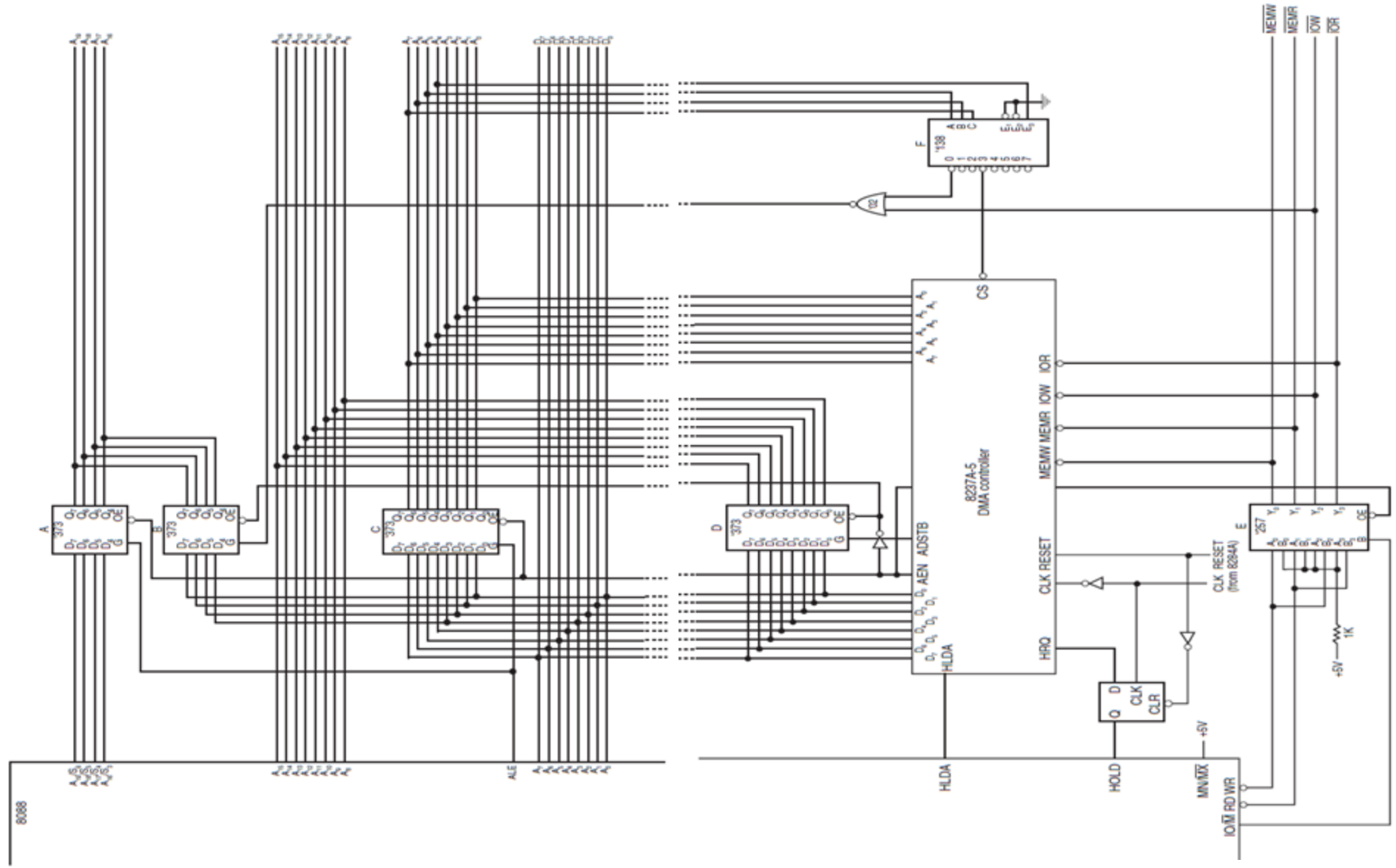
Programming the 8237

- **Four steps** are required to program the 8237:
  - (1) The F/L flip-flop is cleared using a **clear F/L command**
  - (2) the **channel is disabled**
  - (3) LSB & MSB of the address are programmed
  - (4) LSB & MSB of the count are programmed
- Once these **four operations** are performed, the channel is **programmed and ready to use.**
  - additional programming is required to select the mode of operation before the channel is enabled and started

The 8237 Connected to the 80X86

- The address enable (AEN) output of 8237 controls the output pins of the latches and outputs of the 74LS257 (E).
  - during normal operation (AEN=0), latches A & C and the multiplexer (E) provide address bus bits A_{19}–A_{16} and A_7–A_0
- See Figure 13-12.

Figure 13–12 Complete 8088 minimum mode DMA system.



The 8237 Connected to the 80X86

- The multiplexer provides the system control signals as long as the 80X86 is in control of the system.
  - during a DMA action (AEN=1), latches A & C are disabled along with the multiplexer (E)
  - latches D and B now provide address bits A_{19}–A_{16} and A_{15}–A_8
- Address bus bits A_7–A_0 are provided directly by the 8237 and contain part of the DMA transfer address.
- The DMA controller provides control signals.

The 8237 Connected to the 80X86 (*Summary*)

Normal Operation

AEN=0,

A, C, E ⟹ ON

~~A~~ ⟹ A_{19}-A_{16}

Directly ⟹ A_{15}-A_8

~~C~~ ⟹ A_7-A_0

DMA Operation

AEN=1,

B, D ⟹ ON

~~B~~ ⟹ A_{19}-A_{16}

~~D~~ ⟹ A_{15}-A_8

Directly ⟹ A_7-A_0

An 8086 microprocessor system uses the 8237A DMA controller to transfer a block of data. The DMA controller is connected in I/O mapped I/O space with the following port assignments:

- Channel 0 Address Register: 70H
- Channel 1 Address Register: 72H
- Channel 1 Count Register: 73H
- Mode Register: 7BH
- Command Register: 78H
- Request Register: 79H
- Single Mask Register: 7FH
- Page Latch (A16–A19): 10H
- Clear First/Last Flip-Flop: 7CH

The system must transfer 512 bytes of data from I/O port (address 40H) to a memory buffer starting at 2000:1000H. Assume incrementing addresses and single transfer mode.

Solution:

Question:

- Describe step by step how the microprocessor programs the DMA controller registers before the transfer begins.

- After programming, explain how the actual transfer is carried out by the DMA controller until all bytes are moved.

Memory-to-Memory Transfer with the 8237

- Memory-to-memory transfer is much more powerful
- 8237 requires only 2.0 μs per byte, which is over twice as fast as a software data transfer.
- This is not true if an 80386, 80846, or Pentium is in use in the system.

Sample Memory-to-Memory DMA Transfer

- Suppose contents of memory locations 10000H–13FFFH are to be transferred to locations 14000H–17FFFH.
  - accomplished with the DMA controller
- Example 13–1 shows the software required to initialize the 8237 and program latch B in Figure 13–12 for this DMA transfer.

EXAMPLE 13–1

```
;A procedure that transfers a block of data using the 8237A
;DMA controller in Figure 13-12. This is a memory-to-memory
;transfer.

;Calling parameters:
;    SI = source address
;    DI = destination address
;    CX = count
;    ES = segment of source and destination

LATCHB EQU   10H
CLEARF EQU   7CH
CH0A   EQU   70H
CH1A   EQU   72H
CH1C   EQU   73H
MODE   EQU   7BH
CMMD   EQU   78H
MASKS  EQU   7FH
REQ    EQU   79H
STATUS EQU   78H

TRANS PROC   NEAR USES AX

    MOV   AX,ES              ;program latch B
    MOV   AL,AH
    SHR   AL,4
    OUT   LATCHB,AL
    OUT   CLEARF,AL          ;clear F/L

    MOV   AX,ES              ;program source address
    SHL   AX,4
    ADD   AX,SI
    OUT   CH0A,AL
    MOV   AL,AH
    OUT   CH0A,AL

    MOV   AX,ES              ;program destination address
    SHL   AX,4
    ADD   AX,DI
    OUT   CH1A,AL
    MOV   AL,AH
    OUT   CH1A,AL
```

EXAMPLE 13–1 (Cont.)

```
        MOV     AX,CX               ;program count
        DEC     AX
        OUT     CH1C,AL
        MOV     AL,AH
        OUT     CH1C,AL

        MOV     AL,88H              ;program mode
        OUT     MODE,AL
        MOV     AL,85H
        OUT     MODE,AL

        MOV     AL,1                ;enable block transfer
        OUT     CMMD,AL

        MOV     AL,4                ;start DMA
        OUT     REQ,AL

        .REPEAT                     ;wait for completion
                IN  AL,STATUS
        .UNTIL AL & 1
        RET

TRANS ENDP
```

Thank You